

Understanding KV Caching

Accelerating a tiny Shakespeare Transformer on a laptop CPU

Junguang He - MSCS2201 Mini Research Problem

- Motivation: serving more users requires efficient inference. A naive generation loop repeatedly computes the same history.
- Approach: train one small model, then compare generation with and without a request-local KV cache.
- Result: **1.72-2.40x faster model execution** across three prompt lengths, using **158-254 KiB** of final KV tensors.
- Correctness: maximum tested logit difference **0.0000072**. Greedy outputs matched in the tests.

Why inference efficiency matters

Meta projected **\$130-145 billion** in 2026 capital expenditure for AI and its core business. This is company-wide investment, not an inference bill. [4]

Framework	Public deployment example
vLLM	AWS uses vLLM-based optimizations in parts of SageMaker AI and Bedrock model hosting. [5]
SGLang	LinkedIn uses it for LLM ranking in AI Job Search and AI People Search. [6]

Efficient serving can support more requests with the same hardware budget.

Sources: 4: [Meta Q2 2026 filing](#), 5: [AWS engineering](#), 6: [LinkedIn engineering](#).

Why study KV caching?

- A naive decode loop recomputes past token states. KV caching reuses them, reducing computation at the cost of memory.
- Decode, especially at small batch sizes, is often limited by memory bandwidth. Reading weights and historical KV still takes time. [7]
- Production evidence: in 2023, LMSYS reported using **50% fewer serving GPUs** after switching to vLLM. This was a system-level result, not an isolated KV-cache comparison. [8]

Project question: How much model execution time does ordinary KV caching save, and how much cache memory does it use, on a laptop CPU?

Sources: [7: NVIDIA inference guide](#), [8: vLLM deployment report, 2023](#).

Background and related work

Reference	Relevant idea	Role in this project
Vaswani et al. (2017) [1]	Scaled dot-product attention and causal masking	Explains why future characters cannot change historical states
Karpathy's GPT lecture [2]	A small character-level decoder Transformer	Architecture and training reference
Kwon et al. (2023) [3]	KV reuse during generation and cache memory management	Background for caching and its memory cost

This experiment studies ordinary KV caching within one request. It does not implement vLLM's PagedAttention or cross-request prefix caching.

What the cache reuses

Prediction	Recompute history	KV cache
ABC predicts D	Process ABC	Process ABC and save each layer's K, V
ABCD predicts E	Process ABCD	Process only D, using cached K, V from ABC
ABCDE predicts next	Process ABCDE	Process only E, extending the cache

$\text{attention} = \text{softmax}(Q @ K.T / \sqrt{\text{head_size}}) @ V$

- During decode, Q has one new position. K and V cover the whole available history.
- Historical states stay unchanged under causal attention and fixed positions. The new query still attends to historical keys.

Implementation and correctness

```
q, k, v = self.query(x), self.key(x), self.value(x)
if past is not None:
    k = torch.cat((past[0], k), dim=1)
    v = torch.cat((past[1], v), dim=1)
```

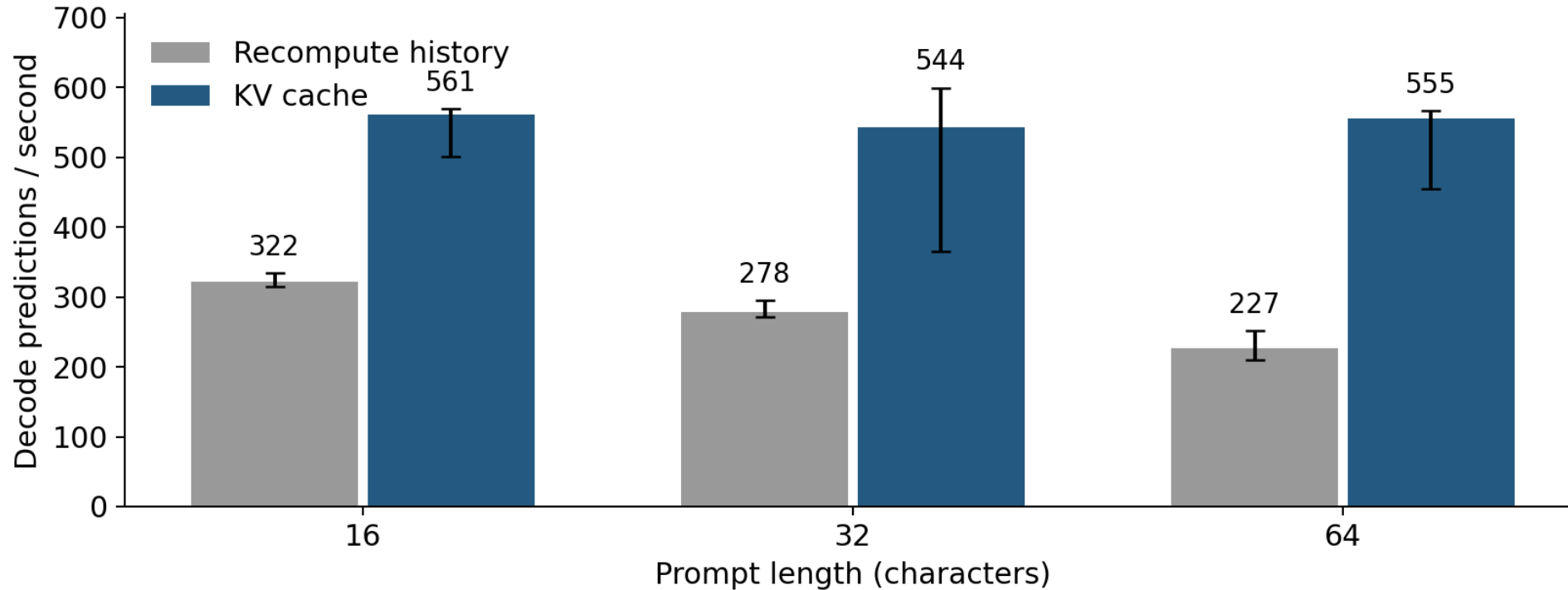
- Continue position numbers from the cache length. Select the matching causal-mask rows.
- Use eval mode and inference mode. Start each request with an empty cache.
- Five checks passed on the trained model, covering logits, greedy output, memory, causality/request isolation and window boundaries.
- Maximum absolute logit difference: **0.0000072**, with absolute tolerance 0.00001 and relative tolerance 0.0001. This is numerical agreement on the tested inputs.

Experimental setup

Item	Setting
Model	2 layers, width 128, 4 heads, 429,121 parameters
Training	Tiny Shakespeare, 90/10 split, 3,000 steps, batch 8
Training outcome	About 5 minutes. Validation loss 4.28 to 1.83 nats/character
Hardware	Ryzen 5 3500U CPU, 4 PyTorch threads, FP32
Workload	Prompts of 16, 32 or 64 characters, 64 output predictions, batch 1

- Both paths replay identical held-out continuations. Two warmups per configuration, then five rounds in shuffled order.
- Timing includes model calls only. Loading, tokenization, sampling and text assembly are excluded. Prefill predicts output #1, followed by 63 decode calls.

Decode speed on this CPU



Bars show medians. Error bars show the full range across five runs.

Cached decode reached about **544-561 predictions/s**, versus **227-322** without caching. The small model and short sequences limit how widely this result can be generalized.

Execution time and cache memory

Prompt	Uncached total	Cached total	Speedup	Final KV size
16 chars	198.3 ms	115.0 ms	1.72x	158 KiB
32 chars	229.6 ms	118.9 ms	1.93x	190 KiB
64 chars	282.1 ms	117.5 ms	2.40x	254 KiB

Totals are medians of prefill + decode model execution time.

- KV bytes = $2 \times \text{layers} \times \text{batch} \times \text{cached positions} \times \text{width} \times 4$. Final cache length is prompt + 63. The last predicted character has not been fed back.
- This implementation copies tensors with cat and requires prompt + output ≤ 128 . KV size excludes temporary allocations and model weights.
- One CPU, one trained model and five timing rounds. Timing outliers remain in the results. These measurements do not establish GPU or production-serving performance.

References and code

- [1] Vaswani et al. (2017). [Attention Is All You Need](#). Sections 3.2 and 3.5.
- [2] Andrej Karpathy. [Let's build GPT: lecture code](#). Architecture reference, reduced model size.
- [3] Kwon et al. (2023). [Efficient Memory Management for Large Language Model Serving with PagedAttention](#). Sections 2.1-2.2 and 3.
- Dataset: [Tiny Shakespeare in char-rnn](#).

Code, raw measurements and reproduction commands:

github.com/junguanghe/sofia-ai-final

Project work: cache implementation and a small CPU experiment.